

The background is a stylized fantasy landscape. In the upper left, a black silhouette of a dragon with spread wings is flying. Several small black birds are scattered across the sky. On the right side, a large, bright yellow sun is partially obscured by a black silhouette of a castle with multiple spires and towers. The foreground consists of dark green and black silhouettes of mountains and trees. The overall color palette is warm, with shades of beige, yellow, and dark green.

Dungeon Pathfinder

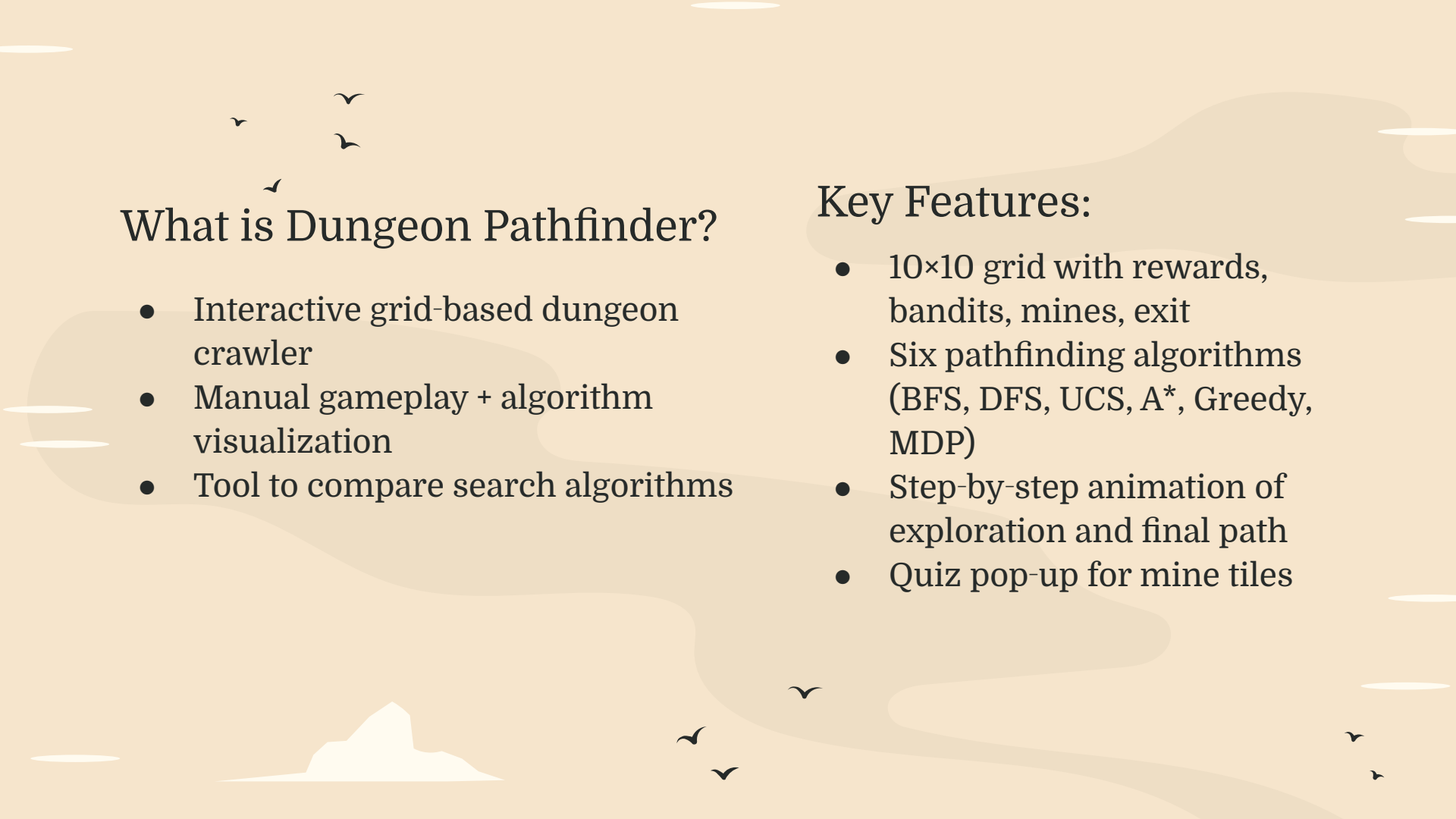
Šehzada Sijarić Farah Piralić Asja Smječanin

[Link For Github](#)

The background is a stylized illustration in a flat, minimalist style. It features a light beige sky with several white, cloud-like shapes. Two large black dragons are depicted in flight, one on the left and one on the right. Several small black birds are scattered across the sky. In the bottom right corner, there is a silhouette of a castle with multiple towers and spires. The bottom left corner shows dark green, jagged shapes representing trees or hills.

01

Introduction

The background is a light beige color with a subtle desert landscape illustration. It includes a white mountain peak at the bottom left, several small black birds in flight scattered across the scene, and soft, wavy beige shapes representing sand dunes or clouds.

What is Dungeon Pathfinder?

- Interactive grid-based dungeon crawler
- Manual gameplay + algorithm visualization
- Tool to compare search algorithms

Key Features:

- 10×10 grid with rewards, bandits, mines, exit
- Six pathfinding algorithms (BFS, DFS, UCS, A*, Greedy, MDP)
- Step-by-step animation of exploration and final path
- Quiz pop-up for mine tiles

Game Mechanics

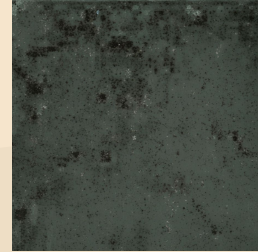
Player



Exit



Empty Tile



Bandit



Mine



Reward



Goal: Collect as much gold as possible before exiting the dungeon



Select Algorithm:

Speed: 100 ms

Select Algorithm...

v

Current Gold

0

Status

Playing

Path Progress

0

Explored Nodes

0

START

PAUSE

STEP

RESET

GENERATE NEW DUNGEON

Algorithm Comparison

Select and run an algorithm to see details

The background is a light beige color with stylized, flat illustrations. On the left, a large black silhouette of a dragon with spread wings is flying. To its right, three small black birds are in flight. On the right side, another large black silhouette of a dragon is flying. In the bottom right corner, there is a black silhouette of a castle with multiple towers and spires. The bottom of the image features dark green, jagged silhouettes of trees or hills. The text '02' is centered in the upper half, and 'Architecture Overview' is centered below it.

02

Architecture Overview

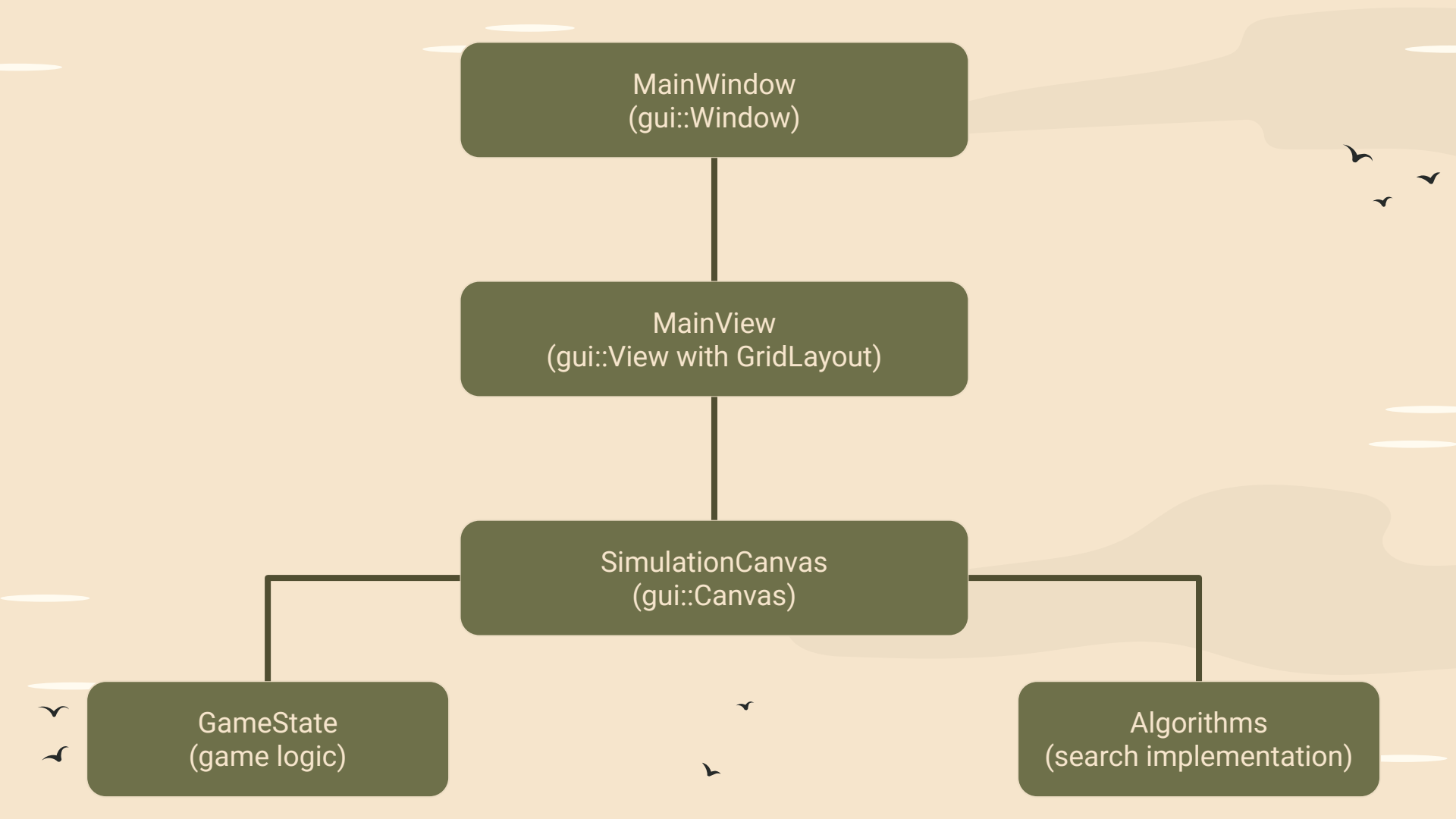
MainWindow
(gui::Window)

MainView
(gui::View with GridLayout)

SimulationCanvas
(gui::Canvas)

GameState
(game logic)

Algorithms
(search implementation)



The background is a stylized, flat-design illustration of a fantasy landscape. It features two large black dragons in flight, one on the left and one on the right. Several small black birds are scattered across the sky. In the bottom right corner, there is a silhouette of a large, multi-towered castle or cathedral. The foreground shows dark green, jagged silhouettes of trees or hills. The sky is a light beige color with some faint, darker beige shapes suggesting clouds or distant land.

03

Core Class - GameState

GameState

Manages:

- actualGrid – true dungeon layout (constants: EMPTY=0, PLAYER=1, etc)
- displayGrid – what the user sees (adds PATH_VISUAL, EXPLORED_NODE)
- Player position, gold, game status

Initialization:

- Randomly places player on left column, exit on right column
- Places 5 rewards, 5 bandits, 5 mines using placeRandomTile

```

bool movePlayer(int newX, int newY) {
    if (gameOver || newX < 0 || newX >= GRID_SIZE || newY < 0 || newY >= GRID_SIZE)
        return false;

    int cellType = actualGrid[newX][newY];

    actualGrid[playerX][playerY] = EMPTY;
    displayGrid[playerX][playerY] = EMPTY;

    playerX = newX;
    playerY = newY;

    if (cellType == REWARD) {
        gold += 10;
        collectedRewards++;
        actualGrid[newX][newY] = PLAYER;
        displayGrid[newX][newY] = REWARD;
        if (gameEventCallback) gameEventCallback("reward", 10);
    }
    else if (cellType == BANDIT) {
        gold = gold / 2;
        actualGrid[newX][newY] = PLAYER;
        displayGrid[newX][newY] = BANDIT;
        if (gameEventCallback) gameEventCallback("bandit", 0);
    }
}

```

```

    else if (cellType == MINE) {
        actualGrid[newX][newY] = PLAYER;
        displayGrid[newX][newY] = MINE;
        if (gameEventCallback) gameEventCallback("mine", 5);
    }
    else if (cellType == EXIT) {
        actualGrid[newX][newY] = PLAYER;
        displayGrid[newX][newY] = PLAYER;
        gameOver = true;
        gameWon = true;
        hasReachedExitOnce = true;
        if (gold >= 20) {
            revealAll();
            if (gameEventCallback) gameEventCallback("exit", 0);
        }
        else {
            if (gameEventCallback) gameEventCallback("exit_insufficient", gold);
        }
    }
    else {
        actualGrid[newX][newY] = PLAYER;
        displayGrid[newX][newY] = PLAYER;
    }

    return true;
}

```

04

Algorithms



Overview

Algorithms:

- BFS – queue, guarantees shortest step count
- DFS – stack, memory efficient but not optimal
- UCS (Dijkstra) – priority queue, uses move cost
- A* – Manhattan heuristic + cost
- Greedy – only heuristic, fast but risky
- MDP – handles uncertainty (70% mine success)

```
// ===== COST FUNCTION =====  
inline int getMoveCost(int cellType) {  
    switch (cellType) {  
        case 2: return 0;  
        case 3: return 15;  
        case 4: return 8;  
        default: return 1;  
    }  
}
```

- All algorithms reside in Algorithms.h
- Return type: SearchResult (path + explored nodes)

Markov Decision Process (MDP)

- Why MDP? Mines have probabilistic outcome (70% success, 30% fail)
- State: (x, y, gold) – gold capped at 50 for tractability
- Value iteration computes expected future reward
- Reward function encourages gold collection and penalizes hazards
- Policy gives best action in every state

```
double getImmediateReward(int cellType, int currentGold, int newGold) const {
    switch (cellType) {
        case 0: // EMPTY
            return -0.05; // Low cost to encourage exploration

        case 1: // PLAYER
            return 0.0;

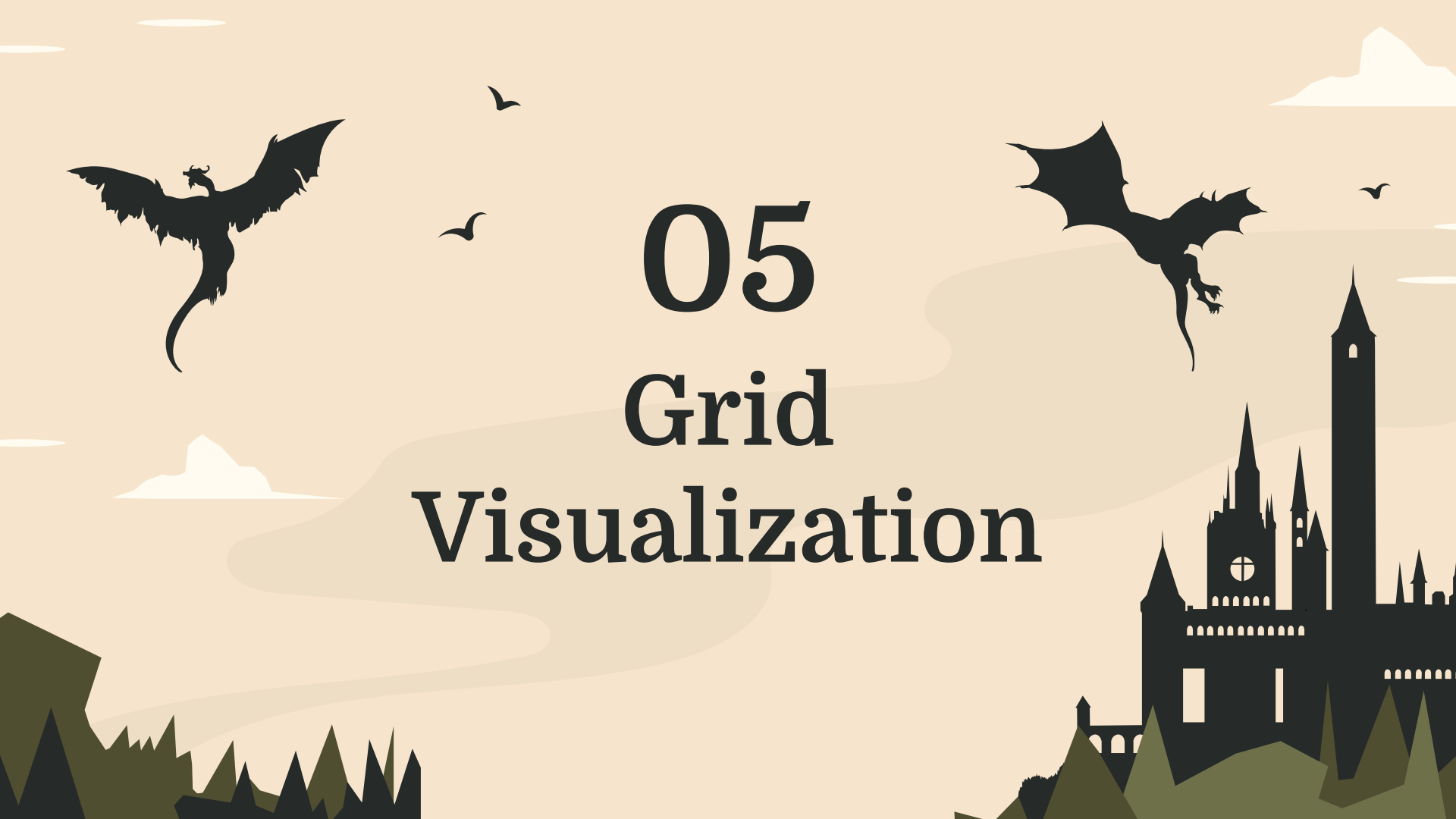
        case 2: // REWARD (+10)
            if (currentGold >= MAX_GOLD_TRACKED) return -0.05;
            return 150.0;

        case 3: // BANDIT
            return -50.0 - (currentGold - newGold) * 5.0;

        case 4: // MINE
            return -10.0;

        case 5: // EXIT
            if (newGold < MIN_GOLD_FOR_WIN) {
                return -10000.0;
            }
            else {
                double extraGold = (double)(newGold - MIN_GOLD_FOR_WIN);
                return 2000.0 + (extraGold * 100.0);
            }

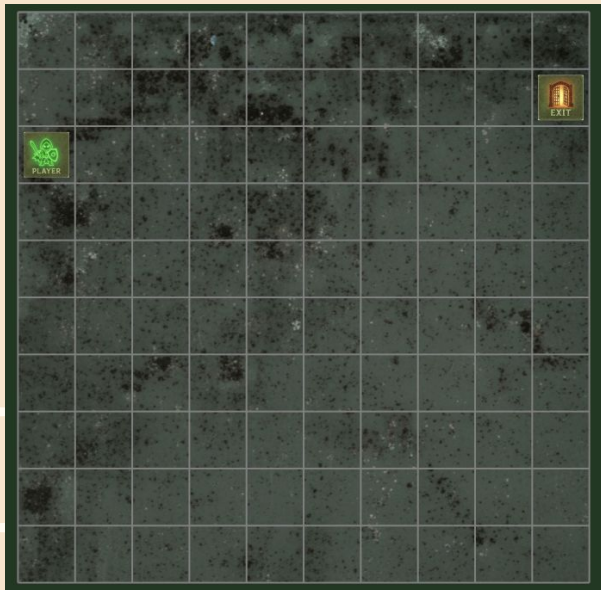
        default:
            return -0.1;
    }
}
```

The background is a stylized, flat-design illustration of a fantasy landscape. It features two large black dragons with spread wings flying in the sky. Several small black birds are also flying. In the bottom right corner, there is a silhouette of a large, multi-towered castle or cathedral. The foreground shows dark green, jagged shapes representing trees or hills. The sky is a light beige color with some white, cloud-like shapes. The overall aesthetic is clean and modern.

05 Grid Visualization



Grid Rendering



- drawGameGrid() splits left area into cells
- Grid lines: gui::Shape with createLines and drawWire
- Cell content: drawCellContent() uses images if available, else backup colors



```

void drawGameGrid() {
    int N = GameState::GRID_SIZE;
    gui::CoordType margin = leftZoneWidth * 0.01;
    gui::CoordType gridSize = leftZoneWidth - 2 * margin;
    gui::CoordType cellSize = gridSize / N;
    gui::CoordType startX = leftZoneLeft + margin;
    gui::CoordType startY = leftZoneTop + margin;

    if (backgroundLoaded) {
        try {
            imgBackground.draw(gui::Rect(startX, startY, startX + gridSize, startY + gridSize));
        }
        catch (...) {
            backgroundLoaded = false;
        }
    }

    if (!backgroundLoaded) {
        gui::Shape gridBg;
        gridBg.createRect(gui::Rect(startX, startY, startX + gridSize, startY + gridSize));
        gridBg.drawFill(td::ColorID::DarkGray);
    }

    for (int i = 0; i <= N; i++) {
        gui::Point pts1[2] = { {startX + i * cellSize, startY}, {startX + i * cellSize, startY + gridSize} };
        gui::Shape line1; line1.createLines(pts1, 2, 2); line1.drawWire(td::ColorID::Gray);

        gui::Point pts2[2] = { {startX, startY + i * cellSize}, {startX + gridSize, startY + i * cellSize} };
        gui::Shape line2; line2.createLines(pts2, 2, 2); line2.drawWire(td::ColorID::Gray);
    }

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            int cellType = algorithmRunning ? displayGrid[i][j] : gameState.getDisplayCell(i, j);
            if (cellType != GameState::EMPTY)
                drawCellContent(startX + i * cellSize, startY + j * cellSize, cellSize, cellType);
        }
    }

    if (isAnimating) {
        gui::Shape border;
        border.createRect(gui::Rect(startX, startY, startX + gridSize, startY + gridSize));
        border.drawWire(td::ColorID::Yellow, 3.0f);
    }
}

```

The background is a light beige color with stylized, flat illustrations. In the upper left, a large black dragon with spread wings is flying. To its right, three small black birds are in flight. In the upper right, another large black dragon is flying. In the bottom right corner, there is a black silhouette of a castle with multiple towers and spires. The bottom left corner shows dark green, jagged shapes representing trees or hills. The overall style is minimalist and graphic.

06

Control Panel

Select Algorithm:

Speed: 100 ms

Fast

Slow

Select Algorithm...



Current Gold

0

Status

Playing

Path Progress

0

Explored Nodes

0

START

PAUSE

STEP

RESET

GENERATE NEW DUNGEON

Algorithm Comparison

Select and run an algorithm to see details

- Built entirely with `gui::Shape` and `gui::DrawableString`
- Dropdown: `gui::Rect` for main button, array of `dropdownItemRects` for items
 - Click toggles `dropdownExpanded`, redraws
- Buttons: `drawButton()` creates rounded rectangles with color indicating enabled state

```

void drawControlPanel() {
    gui::CoordType x = rightZoneLeft;
    gui::CoordType y = rightZoneTop + 15;
    gui::CoordType w = rightZoneWidth;

    gui::CoordType labelW = w * 0.6;
    gui::CoordType speedW = w * 0.31;
    gui::DrawableString::draw("Select Algorithm:", 18,
        gui::Rect(x, y, x + labelW, y + 30),
        gui::Font::ID::SystemNormal, td::ColorID::White, td::TextAlignment::Left, td::VAlignment::Center);
    drawSpeedControl(x + labelW + w * 0.09, y, speedW);
    y += 38;

    gui::CoordType dropdownY = y;
    y += 78;

    drawStatisticsPanel(x, y, w);
    y += 165;

    drawControlButtons(x, y, w);
    y += 190;

    drawComparisonTable(x, y, w);

    drawAlgorithmDropdown(x, dropdownY, w);
}

```

```

void drawControlButtons(gui::CoordType x, gui::CoordType y, gui::CoordType width) {
    gui::CoordType bh = 45, gap = 12, hw = width / 2 - 6;

    startButtonRect = gui::Rect(x, y, x + hw, y + bh);
    pauseButtonRect = gui::Rect(x + hw + 12, y, x + width, y + bh);
    drawButton("START", x, y, hw, bh, td::ColorID::Moss, algorithmRunning && !isAnimating);
    drawButton("PAUSE", x + hw + 12, y, hw, bh, td::ColorID::Moss, algorithmRunning && isAnimating);
    y += bh + gap;

    stepButtonRect = gui::Rect(x, y, x + hw, y + bh);
    resetButtonRect = gui::Rect(x + hw + 12, y, x + width, y + bh);
    drawButton("STEP", x, y, hw, bh, td::ColorID::Moss, algorithmRunning && !isAnimating);
    drawButton("RESET", x + hw + 12, y, hw, bh, td::ColorID::Moss, algorithmRunning);
    y += bh + gap;

    generateNewGameRect = gui::Rect(x, y, x + width, y + bh);
    drawButton("GENERATE NEW DUNGEON", x, y, width, bh, td::ColorID::Copper, true);
}

```

Speed Slider

- Custom vertical slider to control animation speed
- speedControlExpanded toggles visibility
- Drawing: track, fill based on current speed, handle
- Interaction: handleSpeedSliderClick() maps Y coordinate to speed value (10–500 ms)

```
void drawSpeedSlider(gui::CoordType x, gui::CoordType y, gui::CoordType w, gui::CoordType h) {
    speedSliderRect = gui::Rect(x, y, x + w, y + h);

    gui::Shape track; track.createRoundedRect(speedSliderRect, 4);
    track.drawFill(td::ColorID::DarkGray); track.drawWire(td::ColorID::Copper, 1);

    gui::CoordType fillH = (std::min(MAX_SPEED, animationSpeed) * h) / MAX_SPEED;
    if (fillH > 0) {
        gui::Shape fill; fill.createRoundedRect(gui::Rect(x, y, x + w, y + fillH), 4);
        fill.drawFill(td::ColorID::Orange);
    }

    gui::CoordType handleY = y + fillH;
    gui::Shape handle; handle.createRect(gui::Rect(x - 3, handleY - 6, x + w + 3, handleY + 6));
    handle.drawFill(td::ColorID::White); handle.drawWire(td::ColorID::Copper, 2);

    gui::DrawableString::draw("Fast", 4, gui::Rect(x - 15, y - 25, x + w + 15, y - 3),
        gui::Font::ID::SystemSmaller, td::ColorID::LightGray, td::TextAlignment::Center, td::VAlignment::Bottom);
    gui::DrawableString::draw("Slow", 4, gui::Rect(x - 15, y + h + 5, x + w + 15, y + h + 25),
        gui::Font::ID::SystemSmaller, td::ColorID::LightGray, td::TextAlignment::Center, td::VAlignment::Top);
}
```

The background is a stylized illustration in a flat, minimalist style. It features a light beige sky with several white, fluffy clouds. Two large black dragons are depicted in flight, one on the left and one on the right, both with wings spread. Several small black birds are scattered across the sky. In the bottom right corner, there is a silhouette of a large, multi-towered castle or cathedral. The bottom left corner shows dark green, jagged shapes representing trees or hills. The overall aesthetic is clean and modern.

07

Pop Up Quiz

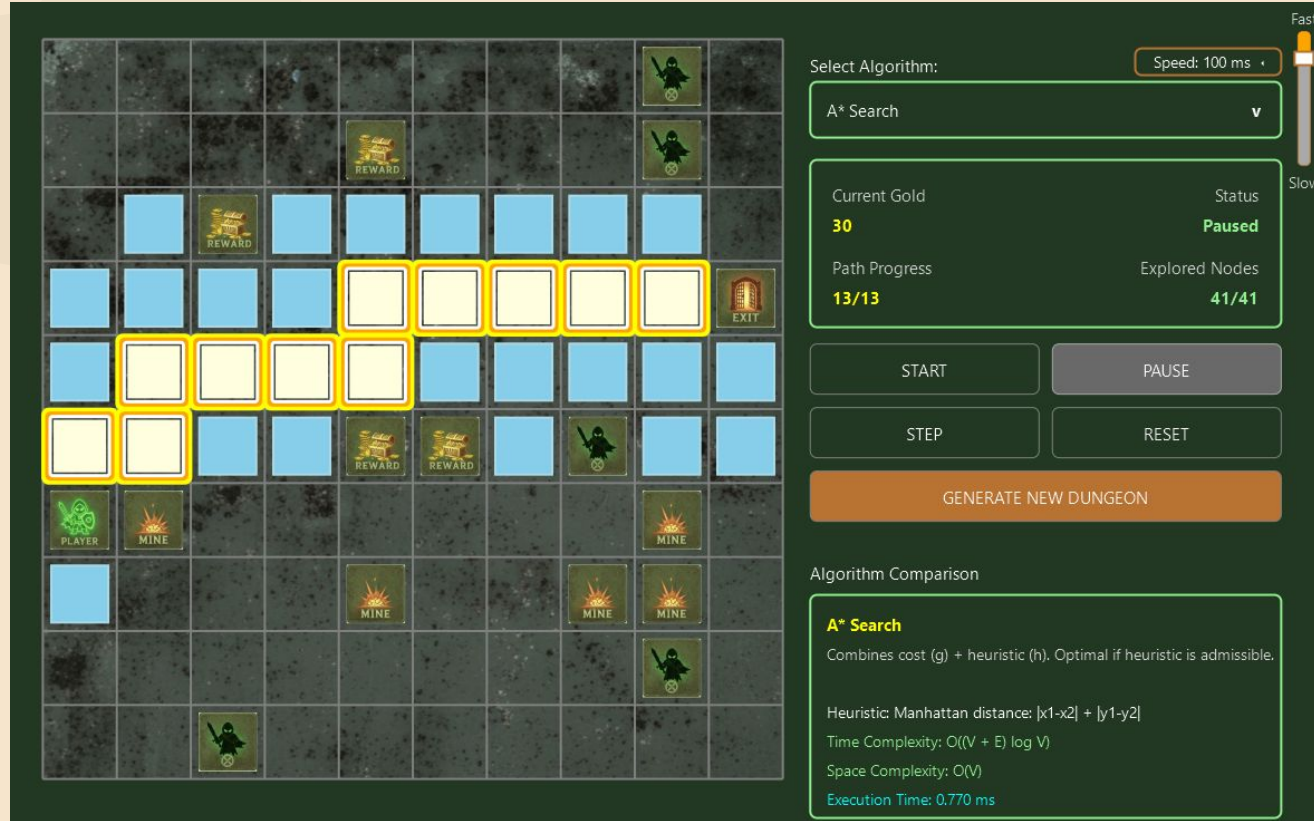
- DialogLogin inherits gui::Dialog
- Three answers mapped to Dialog::Button::ID::OK, Apply, Cancel
- Question database in getAllQuestions() (20 questions on AI topics)
- Random question selected on mine trigger

```
void handleGameEvent(const std::string& event, int value) {
    if (event == "mine") {
        sndMine.play();
        pendingMine.pending = true;
        pendingMine.value = value;
        reDraw();
    }
    else if (event == "reward") {
        sndReward.play();
        td::String message;
        message.format("You found %d gold!\nTotal gold: %d", value, gameState.getGold());
        gui::Alert::show("Reward Found!", message);
    }
    else if (event == "bandit") {
        sndBandit.play();
        td::String message;
        message.format("A bandit stole half your gold!\nRemaining gold: %d", gameState.getGold());
        gui::Alert::show("Bandit Attack!", message);
    }
    else if (event == "exit") {
        sndExit.play();
        td::String message;
        message.format("You escaped the dungeon!\nFinal gold: %d", gameState.getGold());
        gui::Alert::show("You Win!", message);
    }
    else if (event == "exit_insufficient") {
        sndNoExit.play();
        td::String message;
        message.format("You only had %d gold (need 20).\nThe dungeon will reset - try again!", value);
        gui::Alert::show("Cannot Generate New Dungeon", message);
        gameState.resetPlayerPosition();
        reDraw();
    }
}
```

08 Algorithm Animation



- Two-phase animation:
 - Explored nodes (blue tiles) appear one by one
 - Path (yellow glow) is drawn step by step
- Timer-based (animationSpeed) or manual step
- updateAnimation() advances indices and triggers redraw



```

void updateAnimation() {
    if (!isAnimating || !algorithmRunning) return;

    auto now = std::chrono::steady_clock::now();
    auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(now - lastAnimationTime).count();
    if (elapsed < animationSpeed) return;

    if (animationPhase == 0) {
        if (currentExploredIndex < fullExploredNodes.size()) {
            currentExploredIndex++;
            lastAnimationTime = now;
        }
        else {
            animationPhase = 1;
            lastAnimationTime = now;
        }
    }
    else if (animationPhase == 1) {
        if (currentPathIndex < fullAlgorithmPath.size()) {
            currentPathIndex++;
            lastAnimationTime = now;
        }
        else {
            isAnimating = false;
            gui::Canvas::stopAnimation();
        }
    }
}

```

The background is a stylized illustration of a fantasy landscape. It features a light beige sky with soft, wavy horizontal bands of slightly different shades. Two large, black dragon silhouettes are in flight, one on the left and one on the right. Several smaller bird silhouettes are scattered across the sky. In the bottom right corner, there is a black silhouette of a castle with multiple towers and spires. The bottom left corner shows dark green, jagged silhouettes of trees or hills.

09

Event Handling

Keyboard:

onKeyPressed – arrow keys and WASD

```
bool onKeyPressed(const gui::Key& key) override {  
    if (key.isVirtual()) {  
        gui::Key::Virtual k = key.getVirtual();  
        if (k == gui::Key::Virtual::Right) {  
            if (gameState.movePlayer(gameState.getPlayerX() + 1, gameState.getPlayerY())) { playSoundtrack(); reDraw(); }  
            return true;  
        }  
        if (k == gui::Key::Virtual::Left) {  
            if (gameState.movePlayer(gameState.getPlayerX() - 1, gameState.getPlayerY())) { playSoundtrack(); reDraw(); }  
            return true;  
        }  
        if (k == gui::Key::Virtual::Up) {  
            if (gameState.movePlayer(gameState.getPlayerX(), gameState.getPlayerY() - 1)) { playSoundtrack(); reDraw(); }  
            return true;  
        }  
        if (k == gui::Key::Virtual::Down) {  
            if (gameState.movePlayer(gameState.getPlayerX(), gameState.getPlayerY() + 1)) { playSoundtrack(); reDraw(); }  
            return true;  
        }  
    }  
}
```

Mouse:

onPrimaryButtonPressed – checks each interactive rectangle

```
void onPrimaryButtonPressed(const gui::InputDevice& inputDevice) override {  
    gui::Point click = inputDevice.getModelPoint();  
  
    if (speedButtonRect.contains(click)) {  
        speedControlExpanded = !speedControlExpanded;  
        redraw();  
        return;  
    }  
  
    if (speedControlExpanded && speedSliderRect.contains(click)) {  
        handleSpeedSliderClick(click);  
        return;  
    }  
  
    if (dropdownRect.contains(click)) {  
        dropdownExpanded = !dropdownExpanded;  
        redraw();  
        return;  
    }  
}
```

Thank you!

